

VIT-AP UNIVERSITY

ECE3006 Assignment-1

Name: P. Vaasanth

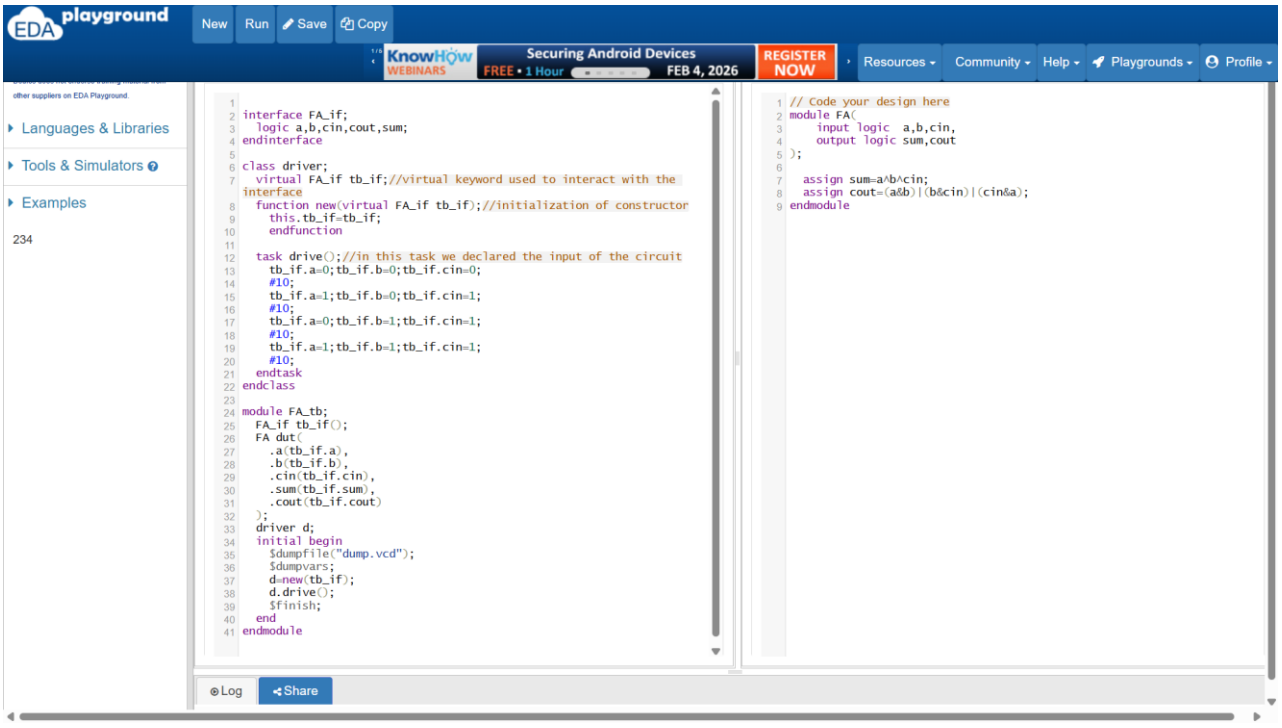
Reg. No: 24BEC7139

Faculty: Dr. Chandan Kumar Pandey

Slot: F2+TF2

1. **(a) Briefly discuss about the role of driver while configuring a layered testbench to verify any DUT. Clearly discuss about input and output flow for "Driver" component.**

Sol.- Driver in the System Verilog is an OOP concept that enable us to generate the input stimuli without writing for repeated number of times. In Driver class we create a task in which required test stimuli is given and we call this task in the testbench module through respective object of this driver class. We can check the multiple designs parallelly by using different objects of the same class which starts at 0ns and are independent of each other.



Output wave form-



In the above example we used the object of driver class 'd' to access the task drive () which generates the output by the given defined input. The data flow is like this it starts from generator in testbench -> driver-> interface-> dut.

(b) Which features of the System Verilog make it preferable over Verilog HDL when verification of a DUT is the main objective? Briefly discuss about them?

Sol.- Traditional Verilog HDL lacks OOPs in it which made System Verilog into wide practice. OOPs and classes simplify the designing and verification of the circuit and this enables programmer to reduce the lines of code by reusability of code. Functional coverage, randomization, property checking, etc. features also give edge for System Verilog over Verilog HDL.

2. **Write a SV program to create a 2D array with the following rows: {14, 10, 15, 3}, {12, 30, 16, 9}, and {7, 9, 15, 10}. Extract elements from this 2D array that are divisible by 3 but not divisible by 2 and store them in a dynamic array. Finally, display the contents of the original 2D array and the resulting dynamic array, using loops and conditional statements for processing.**

Sol-

WEBINARS FREE • 1 Hour MAR 06, 2026

```
module q2;
  int arr[3][4] = '{'{14,10,15,3}, '{12,30,16,9}, '{7,9,15,10}};
  int dyn[$];   // queue

  initial begin
    $display("Original 2D array:");
    foreach(arr[i,j]) $display("arr[%0d][%0d] = %0d", i, j, arr[i][j]);

    foreach(arr[i,j]) begin
      if(arr[i][j] % 3 == 0 && arr[i][j] % 2 != 0) begin
        dyn.push_back(arr[i][j]);
      end
    end

    $display("Queue (div by 3 not 2): %p", dyn);
  end
endmodule
```

Output:

```
Original 2D array:
arr[0][0] = 14
arr[0][1] = 10
arr[0][2] = 15
arr[0][3] = 3
arr[1][0] = 12
arr[1][1] = 30
arr[1][2] = 16
arr[1][3] = 9
arr[2][0] = 7
arr[2][1] = 9
arr[2][2] = 15
arr[2][3] = 10
Queue (div by 3 not 2): '{15, 3, 9, 9, 15}
```

3. **Write a SV program to process the string str1 = "We are appearing for CAT examination". Create two new strings: str2, containing all the vowels, and str3, containing all the consonants from str1, ignoring spaces. Compare the sizes of str2 and str3, and display which string has more characters and the difference in size. Additionally, count and display the total number of spaces in str1.**

Sol-

KnowHow WEBINARS Anatomy of an Embedded Linux System FREE • 1 Hour MAR 06, 2026 REGISTER NOW Resources Community Help

```
module q3;
  string str1 = "We are appearing for CAT examination";
  string str2 = "", str3 = "";
  int spaces = 0, vsize, csize;

  initial begin
    foreach(str1[i]) begin
      string ch;
      ch = string'(str1[i]);

      if(ch == " ")
        spaces++;
      else if(ch inside {"a","e","i","o","u","A","E","I","O","U"})
        str2 = {str2, ch};
      else if(ch inside {"b","c","d","f","g","h","j","k","l","m","n","p","q","r","s","t","v","w","x","y","z",
                         "B","C","D","F","G","H","J","K","L","M","N","P","Q","R","S","T","V","W","X","Y","Z"})
        str3 = {str3, ch};
    end

    vsize = str2.len();
    csize = str3.len();
    $display("------------------------------------------------");
    $display("str2 (vowels): %s, size %0d", str2, vsize);
    $display("str3 (consonants): %s, size %0d", str3, csize);

    if(vsize > csize)
      $display("str2 has more chars, diff %0d ", vsize - csize);
    else
      $display("str3 has more chars, diff %0d ", csize - vsize);

    $display("Spaces in str1: %0d", spaces);
    $display("------------------------------------------------");
    $finish; // Good practice to end the simulation
  end
endmodule
```

Output-

```
------------------------------------------------
str2 (vowels): eaeaeaioAeaiaio, size 15
str3 (consonants): WrpprngfrCTxmntn, size 16
str3 has more chars, diff 1
Spaces in str1: 5
------------------------------------------------
$finish called from file "design.sv", line 33.
```

4. **Write a SV program to assign numbers {3, 6, 4, 5, 2} to five fruits {Apple, Orange, Guava, Grapes, and Mango} using an enumerated data type. Use typedef to give the enumerated data type a new name, fruits. Create a variable of this type and assign Grapes as its current value. Display the values of the first and last members of the enumeration, and also display the name of the current value (i.e., Grapes).**

Sol-

```
module q4;
  typedef enum {Apple=3, Orange=6, Guava=4, Grapes=5, Mango=2} fruits;
  fruits f = Grapes;
  fruits temp;

  initial begin
    temp = f.first();
    $display("-------------------------------------");
    $display("First: %0d (%s)", temp, temp.name());

    temp = f.last();
    $display("Last: %0d (%s)", temp, temp.name());

    $display("Current: %s (%0d)", f.name(), f);
    $display("-------------------------------------");
  end
endmodule
```

Output-

```
Compiler version X-2025.06-SP1_Full64; Runtime v
-------------------------------------
First: 3 (Apple)
Last: 2 (Mango)
Current: Grapes (5)
-------------------------------------
          V C S   S i m u l a t i o n   R e p c
```

5. **(a) Write a SV program to create a queue (q1) of string elements {“AB”, “BC”, “CA”, “CB”, “BA”}. Declare two additional queues (q2 and q3) and store the first two and last three elements of q1 into q2 and q3, respectively. Add two more elements {“CD”, “DC”} to q2 and {“EF”, “FE”} to q3. Finally, display the elements of all three queues, excluding the first element of each queue.**

Sol-

```
module q5;
  string q1[$], q2[$], q3[$];

  initial begin
    q1 = {"AB", "BC", "CA", "CB", "BA"};
    q2 = q1[0:1];
    q3 = q1[2:4];

    q2 = {q2, "CD", "DC"};
    q3 = {q3, "EF", "FE"};
    $display("-----------------------------------------------------");
    $display("q1 (excluding first): %p", q1[1:$]);
    $display("q2 (excluding first): %p", q2[1:$]);
    $display("q3 (excluding first): %p", q3[1:$]);
    $display("-----------------------------------------------------");
  end
endmodule
```

Output-

```
-----------------------------------------------------
q1 (excluding first): '{"BC", "CA", "CB", "BA"}
q2 (excluding first): '{"BC", "CD", "DC"}
q3 (excluding first): '{"CB", "BA", "EF", "FE"}
-----------------------------------------------------
```

(b). With appropriate examples, discuss the difference between packed and unpacked arrays. How size of a dynamic array is increased during run time?

Sol- A packed array is treated as a single, continuous vector of bits. Think of it as a giant integer that you have sliced into smaller parts.

- **Memory:** Guaranteed to be contiguous**.**
- **Usage:** Used for representing registers, buses, or data packets where you need to perform bitwise operations on the whole thing at once.
- **Syntax**: The dimensions are placed before the variable name.

An **unpacked array** is a collection of elements where each element is stored independently. The simulator may add padding between elements for memory alignment.

- Memory: Not guaranteed to be contiguous.
- Usage: Used for modelling RAMs, lookup tables, or collections of objects (like a list of strings).
- Syntax: The dimensions are placed **after** the variable name.

6. Write a program to create a dynamic array containing the first 15 multiples of 7 as its elements. Using the "with" clause and built-in functions, perform the following tasks (use the elements of the created array):

(a) Store the elements, which are greater than 20 but less than 80, into a queue and display it.

(b) Store and display the indices of the elements that are divisible by 5.

(c) Store the indices and values of all odd numbers into two separate queues and display their items.

Sol-

```
module q6;
  int d_array[];
  int q_a[$], q_b_idx[$], q_c_val[$], q_c_idx[$];

  initial begin

    d_array = new[15];
    foreach (d_array[i]) begin
      d_array[i] = (i + 1) * 7;
    end
    $display("Original Array: %p", d_array);
    $display("------------------------------------------------------------");

    q_a = d_array.find(item) with (item > 20 && item < 80);
    $display("(a) Elements > 20 and < 80: %p", q_a);

    q_b_idx = d_array.find_index(item) with (item % 5 == 0);
    $display("(b) Indices of elements divisible by 5: %p", q_b_idx);

    q_c_val = d_array.find(item) with (item % 2 != 0);
    q_c_idx = d_array.find_index(item) with (item % 2 != 0);

    $display("(c) Odd Number Values: %p", q_c_val);
    $display("(c) Odd Number Indices: %p", q_c_idx);
    $display("------------------------------------------------------------");
  end
endmodule
```

Output-

```
Original Array: '{7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84, 91, 98, 105}
------------------------------------------------------------
(a) Elements > 20 and < 80: '{21, 28, 35, 42, 49, 56, 63, 70, 77}
(b) Indices of elements divisible by 5: '{4, 9, 14}
(c) Odd Number Values: '{7, 21, 35, 49, 63, 77, 91, 105}
(c) Odd Number Indices: '{0, 2, 4, 6, 8, 10, 12, 14}
------------------------------------
```

7. **There are two strings given as str1 = “We are students of VIT university” and str2 = “This is HDL verification”. Write a program to count number of vowels in both the strings. Then, using conditional statement, check whether str1 or str2 has more number of vowels. Display the difference between their vowel counts as well.**

Sol-

```
module q7;
  string str1 = "We are students of VIT university";
  string str2 = "This is HDL verification";
  int count1 = 0;
  int count2 = 0;
  int diff;

  initial begin
    foreach (str1[i]) begin
      if (str1[i] inside {"a", "e", "i", "o", "u", "A", "E", "I", "O", "U"}) begin
        count1++;
      end
    end

    foreach (str2[i]) begin
      if (str2[i] inside {"a", "e", "i", "o", "u", "A", "E", "I", "O", "U"}) begin
        count2++;
      end
    end
    $display("------------------------------------------------------------");
    $display("String 1 vowel count: %0d", count1);
    $display("String 2 vowel count: %0d", count2);

    if (count1 > count2) begin
      diff = count1 - count2;
      $display("Result: str1 has more vowels than str2.");
      $display("Difference: %0d", diff);
    end
    else if (count2 > count1) begin
      diff = count2 - count1;
      $display("Result: str2 has more vowels than str1.");
      $display("Difference: %0d", diff);
    end
    else begin
      $display("Result: Both strings have the same number of vowels.");
    end
      $display("------------------------------------------------------------");
  end
endmodule
```

Output-

```
------------------------------------------------------------
tring 1 vowel count: 11
tring 2 vowel count: 8
esult: str1 has more vowels than str2.
ifference: 3
------------------------------------------------------------
```

8. **Write a SV program to declare an enum named “Colours” with the following members and values: red=0, green, blue=4, yellow, white=10, and black. Use a for loop with the first and next methods to display the name and value of each colour. Additionally, implement functionality to check if a specific color (e.g., yellow) exists in the “Colors” enum and display a message indicating whether it exists or not.**

Sol-

```
module q8;

  typedef enum {red=0, green, blue=4, yellow, white=10, black} Colors;

  Colors c;
  string search_name = "yellow";
  bit found = 0;

  initial begin
    $display("--- Iterating through Colors ---");

    c = c.first();

    for (int i = 0; i < c.num(); i++) begin
      $display("Name: %0s, \tvalue: %0d", c.name(), c);

      if (c.name() == "yellow") found = 1;
      c = c.next();
    end

    $display("---------------------------------");

    if (found)
      $display("Check: The color '%0s' exists in the Colors enum.", search_name);
    else
      $display("Check: The color '%0s' does NOT exist.", search_name);
  end
endmodule
```

Output-

```
--- Iterating through Colors ---
Name: red,  value: 0
Name: green,    value: 1
Name: blue,     value: 4
Name: yellow,   value: 5
Name: white,    value: 10
Name: black,    value: 11
---------------------------------
Check: The color 'yellow' exists in the Colors enum.
```

9. **Design a SV program that keeps track of quiz scores for students. Use an associative array where each key is a student's name (string) and the value is their quiz score (int). Traverse the array using a loop to determine and display the names of the students with the highest and lowest scores.**

```
module q9;
  int scores[string];

  string name;
  string max_student, min_student;
  int max_score, min_score;

  initial begin
    scores["Alice"]   = 88;
    scores["Bob"]     = 95;
    scores["Charlie"] = 72;
    scores["Diana"]   = 91;
    scores["Ethan"]   = 84;

    if (scores.first(name)) begin
      max_student = name;
      min_student = name;
      max_score   = scores[name];
      min_score   = scores[name];
    end

    foreach (scores[s]) begin
      if (scores[s] > max_score) begin
        max_score = scores[s];
        max_student = s;
      end

      if (scores[s] < min_score) begin
        min_score = scores[s];
        min_student = s;
      end

      $display("Student: %s, Score: %0d", s, scores[s]);
    end
    $display("----------------------------------------");
    $display("Highest Score: %0d by %s", max_score, max_student);
    $display("Lowest Score:  %0d by %s", min_score, min_student);
    $display("----------------------------------------");
  end
endmodule
```

Output-

```
Student: Alice, Score: 88
Student: Bob, Score: 95
Student: Charlie, Score: 72
Student: Diana, Score: 91
Student: Ethan, Score: 84
----------------------------------------
Highest Score: 95 by Bob
Lowest Score:  72 by Charlie
----------------------------------------
           V C S   S i m u l a t i o n   R e p o r t
```

10. **Write a SV program using qa = {3, 6, 9, 12} and qb = {2, 4, 6, 8}. Create qc using the first 3 elements of qa and last of qb. Insert 50 at the beginning of qc and delete its 3rd element. Create qd by merging qa and qb in reverse, alternating elements. Display all queues after each step.**

Sol-

```
module q10;
  int qa[$] = {3, 6, 9, 12};
  int qb[$] = {2, 4, 6, 8};
  int qc[$], qd[$];

  initial begin
    $display("Initial queues: qa=%p, qb=%p", qa, qb);

    qc = {qa[0:2], qb[$]};
    $display("Step 1 (qc created): %p", qc);

    qc.push_front(50);
    $display("Step 2 (Insert 50): %p", qc);

    qc.delete(2);
    $display("Step 3 (Delete 3rd): %p", qc);

    for (int i = 3; i >= 0; i--) begin
      qd.push_back(qb[i]);
      qd.push_back(qa[i]);
    end
    $display("Step 4 (qd alternating reverse): %p", qd);

    $display("----------------------------------------");
    $display("Final qa: %p", qa);
    $display("Final qb: %p", qb);
    $display("Final qc: %p", qc);
    $display("Final qd: %p", qd);
    $display("----------------------------------------");
  end
endmodule
```

Output-

```
Initial queues: qa='{3, 6, 9, 12} , qb='{2, 4, 6, 8}
Step 1 (qc created): '{3, 6, 9, 8}
Step 2 (Insert 50): '{50, 3, 6, 9, 8}
Step 3 (Delete 3rd): '{50, 3, 9, 8}
Step 4 (qd alternating reverse): '{8, 12, 6, 9, 4, 6, 2, 3}
----------------------------------------
Final qa: '{3, 6, 9, 12}
Final qb: '{2, 4, 6, 8}
Final qc: '{50, 3, 9, 8}
Final qd: '{8, 12, 6, 9, 4, 6, 2, 3}
----------------------------------------
          V C S   S i m u l a t i o n   R e p o r t
```